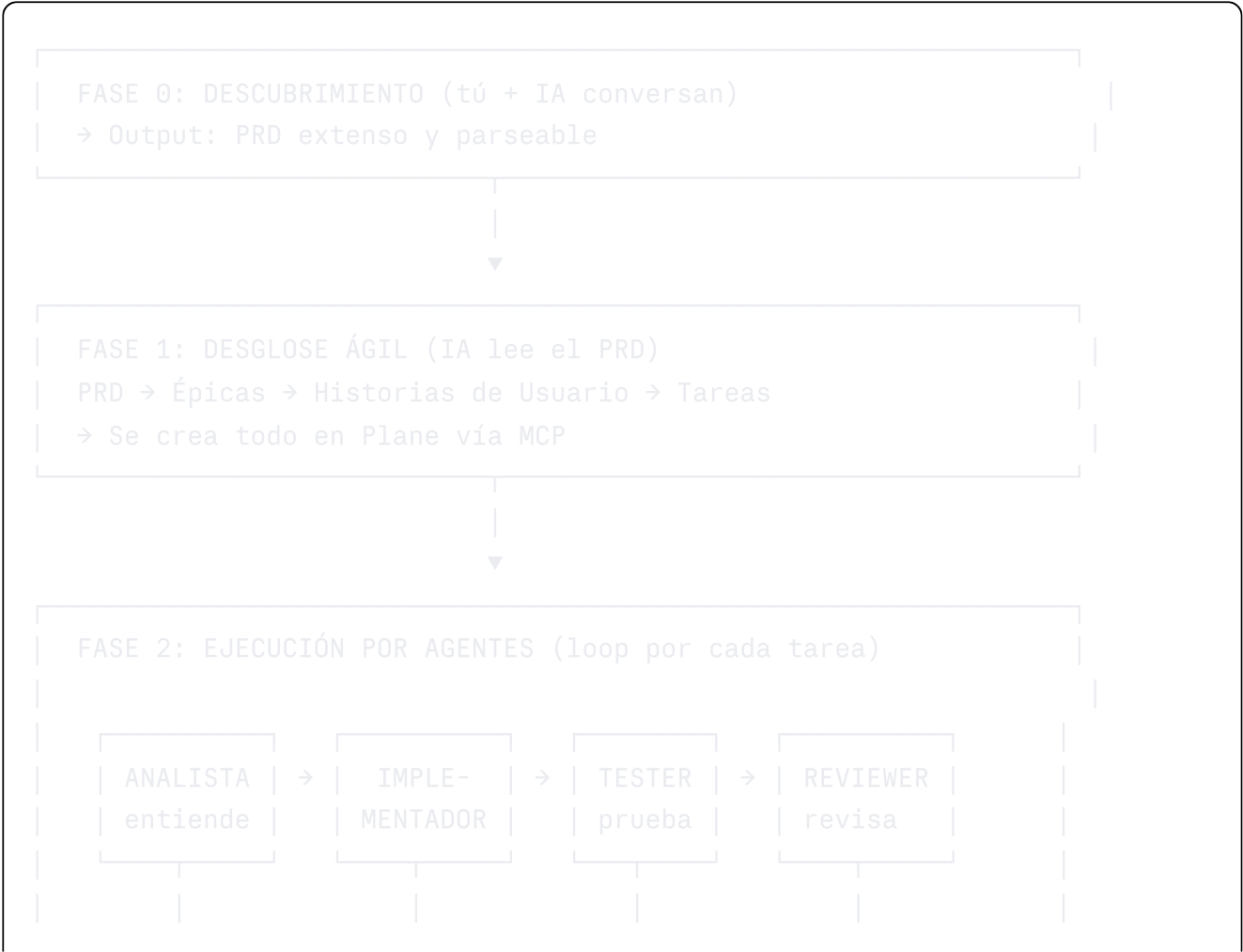


Sistema de Pipeline: PRD → Plane → Agentes → Git

Stack: Antigravity IDE · IA (Claude/agentes) · Plane (MCP oficial, plan Free) · Git
Modo: Semi-autónomo (tú apruebas cada gate) **Filosofía:** El PRD es la única fuente de verdad. Todo lo demás se desglosa de él.

1. Arquitectura general





Por qué cada pieza:

- **Plane** maneja la jerarquía ágil nativa: Initiatives → Projects → Epics → Cycles (sprints) → Work Items. No tienes que inventar el sistema de tracking.
- **MCP oficial de Plane** (alojado en <https://mcp.plane.so>) expone toda la API como herramientas. Los agentes en Antigravity leen y escriben work items directamente.
- **Git** es el gate de calidad final. Cada historia = una branch = un PR. El work item de Plane se vincula al PR y cambia de estado automáticamente al mergear.
- **Antigravity** es donde corren tus agentes (no consumen los 500 créditos IA del plan Free de Plane, porque esos son solo para la IA *nativa* de Plane).

2. Setup inicial (una sola vez)

2.1 Conectar el MCP de Plane a Antigravity

1. En Plane: **Profile Settings** → **API Tokens** → genera un Personal Access Token. Cópialo (no se vuelve a mostrar).
2. Anota tu **workspace slug** (el identificador corto en la URL de Plane: plane.so/<slug>/...)).
3. En la config MCP de Antigravity (formato estándar de cualquier cliente MCP):

json

```
{
  "mcpServers": {
    "plane": {
      "url": "https://mcp.plane.so/http/api-key/mcp",
      "type": "http",
      "headers": {
        "x-api-key": "TU_API_KEY_AQUI",
        "x-workspace-slug": "tu-workspace-slug"
      }
    }
  }
}
```

Si Antigravity solo soporta MCP por subprocesso local (stdio), usa:

json

```
{ "mcpServers": { "plane": {
  "command": "uvx",
  "args": ["plane-mcp-server", "stdio"],
  "env": {
    "PLANE_API_KEY": "TU_API_KEY",
    "PLANE_WORKSPACE_SLUG": "tu-slug",
    "PLANE_BASE_URL": "https://api.plane.so"
  }
}
}}
```

2.2 Integrar Git con Plane

En Plane: **Workspace Settings** → **Integrations** → conecta GitHub o GitLab. Esto hace que

cada work item pueda crear su branch, vincular el PR, y cambiar de estado solo al mergear.

2.3 Configurar los estados (workflow) en Plane

Crea estos estados personalizados en tu proyecto para que reflejen el paso por cada agente:

Estado	Significado	Quién lo mueve
Backlog	Tarea creada, sin refinar	IA (desglose)
Refinada	Analista la entendió y planeó	Agente Analista
En desarrollo	Implementador trabajando	Agente Implementador
En pruebas	Tester validando	Agente Tester
En revisión	PR abierto, Reviewer revisando	Agente Reviewer / Git
Hecho	PR mergeado, criterios cumplidos	Git (auto)
Bloqueada	Necesita tu intervención	Cualquier agente

3. Plantilla de PRD parseable

Regla de oro: El PRD NO es prosa libre. Tiene secciones estructuradas con IDs estables, para que la IA lo desglose automáticamente sin ambigüedad. Guárdalo como PRD.md en la raíz del repo.

markdown

PRD: [Nombre del Proyecto]

META

1. VISIÓN Y PROBLEMA

2. USUARIOS Y ROLES

1

ID

1

Ro1

1

Descripción

1

Necesidad principal

1

1

.....

1

1

1

|
U-01

|
[rol]

|
[quién es]

|
[qué necesita]

|
U-02

|
[rol]

|
...

|
...

3. OBJETIVOS Y MÉTRICAS DE ÉXITO

|
ID

|
Objetivo

|
Métrica

|
Meta

|

```
-----  
|  
-----  
|  
-----  
|  
|  
0-01  
|  
[objetivo de negocio]  
|  
[cómo se mide]  
|  
[valor]  
|  
|  
0-02  
|  
...  
|  
...  
|  
...  
|
```

4. ALCANCE

Dentro del alcance (MVP)

- [funcionalidad 1]
- [funcionalidad 2]

Fuera del alcance (por ahora)

- [lo que NO se hace en esta versión]

5. ÉPICAS

> Cada épica es un gran bloque de valor. Se mapea a un Epic/Module en Plane.

EP-01: [Nombre de la épica]

- **Objetivo:** [qué logra esta épica]
- **Vinculada a:** 0-01 (objetivo)
- **Prioridad:** Alta | Media | Baja

EP-02: [Nombre de la épica]

- ...

6. HISTORIAS DE USUARIO

> Formato estándar. Cada historia se mapea a un Work Item (Issue) en Plane.

> Cada historia pertenece a UNA épica.

HU-001 (Épica: EP-01)

- **Historia:** Como [rol U-01] quiero [acción] para [beneficio].
- **Prioridad:** Alta | Media | Baja
- **Estimación:** [story points: 1,2,3,5,8,13]
- **Criterios de aceptación:**
 - [] DADO [contexto] CUANDO [acción] ENTONCES [resultado esperado]
 - [] DADO ... CUANDO ... ENTONCES ...
- **Notas técnicas:** [restricciones, dependencias, APIs a usar]
- **Dependencias:** [HU-XXX que deben hacerse antes, si aplica]

HU-002 (Épica: EP-01)

- ...

7. REQUISITOS NO FUNCIONALES

|
ID
|

Tipo

Requisito

RNF-01

Rendimiento

[ej: respuesta < 200ms]

RNF-02

Seguridad

[ej: autenticación OAuth2]

RNF-03

Escalabilidad

...

8. RESTRICCIONES TÉCNICAS

- ****Stack:**** [lenguajes, frameworks, DB]
- ****Infra:**** [dónde se despliega]
- ****Integraciones:**** [servicios externos]

Por qué este formato funciona: cada elemento tiene un ID (EP-01, HU-001, O-01). La IA puede parsear "todas las HU de EP-01", validar que cada historia tenga criterios de aceptación, y detectar dependencias. Sin IDs estables, el desglose automático se vuelve frágil.

4. Estructura en Plane

Mapeo directo del PRD a la jerarquía de Plane:

Elemento del PRD	Objeto en Plane	Notas
Proyecto completo	Project	Un proyecto por PRD
Visión/Objetivos (O-XX)	Initiative (opcional)	Marca la dirección estratégica
Épica (EP-XX)	Epic o Module	Agrupar historias
Historia (HU-XXX)	Work Item / Issue	La unidad de trabajo principal
Tarea (desglose técnico)	Sub-issue	Pasos concretos de cada historia
Sprint	Cycle	Ventana de tiempo (ej: 2 semanas)
Story points	Estimate	Campo nativo de Plane
Criterios de aceptación	Descripción del issue	En formato checklist

Organización de sprints (Cycles)

1. Crea un **Cycle** por sprint (ej: "Sprint 1 — 2 semanas").
 2. Asigna historias al cycle según prioridad y capacidad (suma de story points).
 3. Regla práctica para empezar solo: no metas más de ~13-20 story points por sprint hasta que conozcas tu velocidad real.
-

5. Fase 1 — Prompt de desglose (IA lee el PRD)

Este es el prompt que le das a la IA en Antigravity para que convierta el PRD en items de Plane:

ROL: Eres un Product Owner técnico experto en metodología ágil.

ENTRADA: El archivo PRD.md adjunto.

TAREA:

1. Lee el PRD completo. Identifica todas las Épicas (EP-XX) y todas las Historias de Usuario (HU-XXX).
2. Valida cada historia: ¿tiene criterios de aceptación claros y testeables?
¿tiene estimación? Si algo falta o es ambiguo, NO inventes: mácalo en una lista de "DUDAS PARA EL OWNER" y detente para preguntarme.
3. Una vez validado, usa las herramientas MCP de Plane para crear:
 - a. Un Epic/Module por cada EP-XX.
 - b. Un Work Item por cada HU-XXX, vinculado a su épica, con:
 - Título: el resumen de la historia
 - Descripción: la historia completa + criterios de aceptación como checklist
 - Estimate: los story points
 - Estado inicial: "Backlog"
 - Prioridad: la del PRD
 - c. Sub-issues para el desglose técnico de cada historia (si lo conoces; si no, déjalo para el agente Analista).
4. Detecta dependencias entre historias y anótalas en cada work item.
5. Devuélveme un resumen: cuántas épicas, cuántas historias, total de story points, y propuesta de asignación a los primeros 1-2 sprints.

REGLA: No crees nada en Plane hasta que yo apruebe el resumen del paso 5.

[GATE 0] — Tú revisas el resumen antes de que se cree nada en Plane.

6. Fase 2 — Los 4 agentes (loop por tarea)

Cada agente es un prompt de sistema que corres en Antigravity. El loop procesa **una historia a la vez**.

Agente 1 — ANALISTA

ROL: Eres un analista técnico. Tu trabajo es ENTENDER una historia de usuario, no implementarla.

ENTRADA: Una historia (HU-XXX) que leerás de Plane vía MCP, y el código del repo.

TAREA:

1. Lee la historia y sus criterios de aceptación desde Plane.
2. Lee el código relevante del repo para entender el contexto actual.
3. Verifica que la historia esté COMPLETA y SIN AMBIGÜEDAD. Si falta info, cambia el estado a "Bloqueada" en Plane, deja un comentario con tus dudas, y detente.
4. Si está completa, produce un PLAN TÉCNICO:
 - Archivos que se van a crear/modificar
 - Pasos concretos de implementación (como sub-tareas)
 - Tests que harán falta para cubrir los criterios de aceptación
 - Riesgos o decisiones técnicas que requieren mi opinión
5. Escribe el plan como comentario en el work item de Plane y crea las sub-issues correspondientes.
6. Cambia el estado a "Refinada".

REGLA: NO escribas código. Solo análisis y plan.

[GATE 1] — Tú revisas el plan técnico. ¿Tiene sentido el enfoque? ¿Falta algo?
Apruebas o pides ajustes.

Agente 2 — IMPLEMENTADOR

ROL: Eres un desarrollador senior. Implementas exactamente lo que el plan técnico aprobado especifica.

ENTRADA: La historia (HU-XXX) y su plan técnico aprobado, desde Plane.

TAREA:

1. Lee el plan técnico y los criterios de aceptación desde Plane.
2. Crea una branch siguiendo el formato: feature/HU-XXX-descripcion-corta
3. Implementa el código siguiendo el plan. Sigue las convenciones del repo.
4. Haz commits atómicos y descriptivos, referenciando la historia:
"feat(HU-XXX): descripción del cambio"
5. NO escribas los tests (eso es del Tester), pero asegúrate de que el código
sea testeable.
6. Cuando termines, cambia el estado a "En desarrollo" → marca como listo para
pruebas, y deja un comentario en Plane resumiendo qué implementaste.

REGLA: Cíñete al plan. Si descubres que el plan está mal o incompleto, NO improvises grandes cambios: marca la historia como "Bloqueada", explica el problema, y detente.

[GATE 2] — Tú das un vistazo rápido al código antes de que pase a pruebas (opcional pero recomendado al principio).

Agente 3 — TESTER

ROL: Eres un ingeniero de QA. Validas que la implementación cumpla los criterios de aceptación.

ENTRADA: La historia (HU-XXX), su código implementado, y los criterios de aceptación desde Plane.

TAREA:

1. Lee los criterios de aceptación desde Plane.
2. Escribe tests automatizados que cubran CADA criterio de aceptación.
3. Corre toda la suite de tests (los nuevos y los existentes para detectar regresiones).
4. Documenta los resultados:
 - Criterios cumplidos ✓ / no cumplidos ×
 - Tests que pasan / fallan
 - Cobertura
5. Si algo falla, cambia el estado a "En desarrollo" otra vez, deja un comentario en Plane con el detalle del fallo, para que el Implementador lo corrija.
6. Si todo pasa, cambia el estado a "En pruebas" → listo para revisión, commit
de los tests en la misma branch, y comenta el reporte en Plane.

REGLA: Un criterio de aceptación sin test que lo verifique = criterio NO cumplido. Sé estricto.

[GATE 3] — Tú revisas el reporte de tests. ¿Cubre realmente los criterios? ¿La cobertura es aceptable?

Agente 4 — REVIEWER

ROL: Eres un revisor de código senior. Haces code review en el PR antes del merge.

ENTRADA: La branch feature/HU-XXX-... con el código y los tests.

TAREA:

1. Abre un Pull Request desde la branch hacia main/develop. En la descripción
del PR, referencia el work item de Plane (esto lo vincula automáticamente).
2. Revisa el código contra:
 - Cumplimiento del plan técnico
 - Calidad: legibilidad, convenciones, sin código muerto
 - Seguridad: sin secretos hardcodeados, validación de inputs
 - Los tests realmente cubren los criterios de aceptación
3. Deja comentarios línea por línea en el PR para cualquier problema.
4. Veredicto:
 - APROBADO → comenta en Plane y notifícame que está listo para mi aprobación final y merge.
 - CAMBIOS SOLICITADOS → estado "En desarrollo", lista de cambios en el PR
y en Plane.

REGLA: NO mergees tú. El merge final lo apruebo yo (modo semi-autónomo).

[GATE 4 — FINAL] — Tú das la aprobación final y haces el merge (o autorizas al agente a mergear). Al mergear, Plane mueve la historia a "Hecho" automáticamente.

7. Flujo Git por historia

```
1. Implementador crea:   feature/HU-XXX-descripcion
2. Commits:             feat(HU-XXX): ... | test(HU-XXX): ...
3. Reviewer abre PR:    "HU-XXX: [título]" + link al work item de Plane
4. Tú apruebas + merge → Plane cierra la historia automáticamente
5. Branch se borra. Sprint avanza.
```

Convenciones:

- **Branch:** `feature/HU-XXX-descripcion-corta` (o `fix/`, `chore/`)
 - **Commit:** Conventional Commits con el ID de la historia → `feat(HU-001): añade login con OAuth`
 - **PR:** título empieza con el ID de la historia; descripción enlaza el work item de Plane.
-

8. Cómo arrancar (orden recomendado)

1. **Setup** (sección 2): conecta MCP de Plane a Antigravity + integra Git con Plane + crea los estados del workflow.
2. **Descubrimiento** (Fase 0): conversa con la IA hasta tener el PRD completo usando la plantilla de la sección 3.
3. **Desglose** (Fase 1): corre el prompt de la sección 5. Aprueba el resumen → se crea todo en Plane.
4. **Primer sprint:** asigna 1-2 historias a un Cycle. No más de ~13 story points hasta conocer tu velocidad.
5. **Loop de agentes** (Fase 2): toma la primera historia y pásala por Analista → Implementador → Tester → Reviewer, aprobando cada gate.

6. **Itera:** cierra la historia con el merge, toma la siguiente. Al cerrar el sprint, haz una retro rápida y ajusta.
-

9. Consejos para que no se rompa

- **El PRD manda.** Si un agente quiere desviarse del PRD, debe bloquear y preguntar, nunca improvisar.
- **Empieza con gates estrictos.** Al principio revisa cada gate de verdad. Cuando confíes en el pipeline, puedes relajar GATE 2.
- **Una historia a la vez** hasta que el flujo sea sólido. Luego puedes paralelizar historias sin dependencias.
- **Comentarios en Plane = bitácora.** Cada agente deja su rastro en el work item. Eso te da trazabilidad total: quién entendió qué, qué se probó, qué se revisó.
- **Story points para velocidad, no para presión.** Sirven para saber cuánto cabe en un sprint, no para apurar.
- **Los 500 créditos IA del plan Free** son solo para la IA nativa de Plane. Tus agentes en Antigravity no los tocan, así que no te preocupes por agotarlos con este flujo.